

SIMATS ENGINEERING

ASSESSMENT TOOL 3

Name : C.Manohar
Reg No : 192372246
COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

QUESTIONS:

Total Marks: 20

Question 1 (5 Marks)

Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.

Question 2 (5 Marks)

Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs

such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.

Question 3 (5 Marks)

Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.

Question 4 (5 Marks)

Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.

Code of Conduct:

I C.Manohar / 192372246 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Manohar

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ANSWERS

Question 1

A company wants its website to work seamlessly across mobile, tablet, and desktop devices. This requires responsive web design supported by CSS Flexbox and Grid for adaptive layouts.

1. Concept of Responsive Web Design and Its Importance

Responsive Web Design (RWD) is an approach to web development in which a single website automatically adjusts its layout, images, and content presentation based on the screen size, resolution, and orientation of the device being used. Instead of building separate versions of a site for mobile, tablet, and desktop, RWD uses flexible grids, flexible images, and CSS media queries so that one codebase renders correctly everywhere.

Importance: it improves user experience by avoiding horizontal scrolling and tiny unreadable text; it is cost-effective since only one site needs to be built and maintained; it improves SEO because search engines favour responsive, mobile-friendly sites; and it future-proofs the site against new device sizes.

2. Key CSS Techniques Used to Achieve Responsiveness

- Fluid Grid Layouts - widths defined in percentages (%) or fr units instead of fixed pixels.
- Flexible Images/Media - max-width: 100%; height: auto; so images never overflow their container.
- Media Queries - @media rules apply different CSS rules based on viewport width, height, or orientation.
- Relative Units - rem, em, vw, vh instead of fixed px, so sizing scales with the viewport.
- Mobile-First Design - base styles for small screens first, then min-width queries add styles for larger screens.
- CSS Flexbox and CSS Grid - modern layout systems that redistribute space automatically as screen size changes.

3. Using Flexbox and Grid for Adaptive Layouts

Flexbox (display: flex) is one-dimensional - it arranges items in a row or column and suits components such as navigation bars or card rows. Its properties (flex-wrap, justify-content, align-items) let items wrap and resize as space changes.

CSS Grid (display: grid) is two-dimensional - it controls rows and columns together, making it ideal for full-page layouts (header, sidebar, main, footer). repeat() and minmax() with auto-fit/auto-fill let a grid add or remove columns automatically based on available width.

4. Example Layout

```
<div class="container">
  <header>Site Header</header>
  <nav>Home / About / Contact</nav>
  <main>Main Content</main>
  <aside>Sidebar</aside>
  <footer>Footer</footer>
```

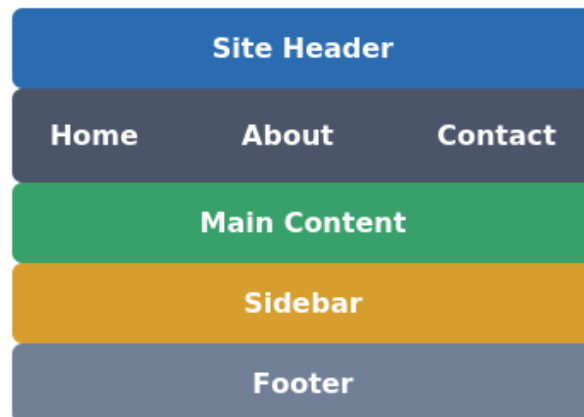
```

</div>

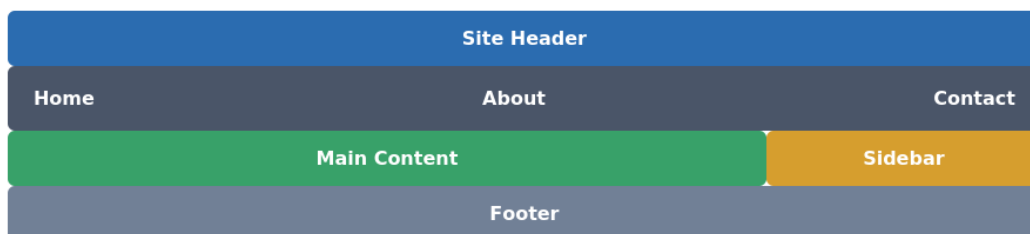
.container { display: flex; flex-wrap: wrap; gap: 12px; }
header, nav, footer { flex: 1 1 100%; }
main, aside { flex: 1 1 100%; }
@media (min-width: 768px) {
  main { flex: 3 1 0; }
  aside { flex: 1 1 0; }
}

```

Output - Mobile screen (below 768px): stacked in a single column



Output - Tablet/Desktop screen (768px and above): Main and Sidebar sit side by side



Grid/Flexbox controls the overall page skeleton (single column on mobile, two-column on tablet/desktop), while the nav links wrap neatly on narrow screens.

5. Suggested Improvements for Better User Experience

- Use CSS clamp() for typography so font size scales smoothly instead of jumping at breakpoints.
- Add <meta name="viewport" content="width=device-width, initial-scale=1"> for correct mobile scaling.
- Use responsive images with srcset/sizes so smaller devices download smaller files.
- Test at multiple breakpoints (320px, 768px, 1024px, 1440px) using browser DevTools.
- Keep touch targets at least 44x44px on mobile for accessibility.

Question 2

A webpage contains a form whose input must be validated before submission. This requires client-side validation using JavaScript event handling and regular expressions.

1. Concept and Purpose of Client-Side Validation

Client-side validation checks form data for correctness in the browser, using JavaScript, before it is sent to the server. Its purpose is to give immediate feedback, catch obvious mistakes early (empty fields, wrong format, mismatched passwords), and reduce unnecessary server requests, improving both user experience and server efficiency.

2. Role of JavaScript Event Handling in Validation

- submit - fires on the form; used with `event.preventDefault()` to stop submission until validation passes.
- input / keyup - fires as the user types, enabling real-time validation feedback.
- blur / focusout - fires when a field loses focus, validating one field at a time.
- change - fires when a select or checkbox value changes.

Listeners are typically attached with `addEventListener()` rather than inline HTML attributes, keeping behaviour separate from markup.

3. Role of Regular Expressions in Input Validation

A regular expression (regex) is a pattern used to test whether a string matches a specific format. It is essential for structured inputs such as email addresses and phone numbers, where simple length or emptiness checks are not enough. The `test()` method returns true or false based on whether the input matches the pattern.

4. Example

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const phoneRegex = /^[6-9]\d{9}$/;

form.addEventListener('submit', function (event) {
    event.preventDefault();
    let valid = true;

    const email = document.getElementById('email').value.trim();
    if (!emailRegex.test(email)) {
        document.getElementById('emailError').textContent = 'Enter a valid email';
        valid = false;
    }

    const phone = document.getElementById('phone').value.trim();
    if (!phoneRegex.test(phone)) {
        document.getElementById('phoneError').textContent = 'Enter a valid 10-digit number';
        valid = false;
    }

    if (valid) form.submit();
});
```

Output - Email "john@" and Phone "12345" submitted (both invalid)

Email

Enter a valid email

Phone

Enter a valid 10-digit phone number

5. Suggested Improvements for Usability and Security

- Use live validation on input/blur events so errors show immediately, not just after submit.
- Show clear, specific error messages near each field instead of a generic alert.
- Never rely on client-side validation alone for security - always repeat validation on the server.
- Use HTML5 attributes (required, type="email", pattern) as a first layer, JS as enhancement.
- Sanitize and escape input on the server to prevent XSS/SQL injection.

Question 3

A webpage layout breaks when the browser window is resized. Understanding the CSS box model, positioning techniques, and media queries helps diagnose and fix this.

1. CSS Box Model and Its Components

- Content - the actual text/image, sized by width and height.
- Padding - transparent space between content and border; increases visible size.
- Border - a line surrounding padding and content.
- Margin - transparent space outside the border, separating elements.

By default (box-sizing: content-box), width/height apply only to content, so padding and border add extra size - a common cause of overflow. box-sizing: border-box makes width/height include padding and border, which is far more predictable for responsive layouts.

2. How CSS Positioning Affects Layout Structure

- static - default flow; top/left/right/bottom have no effect.
- relative - stays in flow but can be shifted; creates a positioning context for absolute children.
- absolute - removed from normal flow, positioned relative to nearest positioned ancestor; can cause overlap.
- fixed - positioned relative to the viewport, stays in place while scrolling.
- sticky - relative until a scroll threshold, then behaves like fixed.

3. Common Causes of Layout Breaking on Resize

- Fixed pixel widths that don't shrink on smaller screens, forcing horizontal scroll.
- Ignoring box-sizing, so padding/border push elements wider than intended.
- Overuse of absolute/fixed positioning, causing overlap at different viewport sizes.
- Images without max-width: 100%, overflowing their parent container.
- Uncleared floats, collapsing parent containers.
- Missing viewport meta tag, causing mobile browsers to render the desktop layout zoomed out.

4. How Media Queries Resolve These Issues

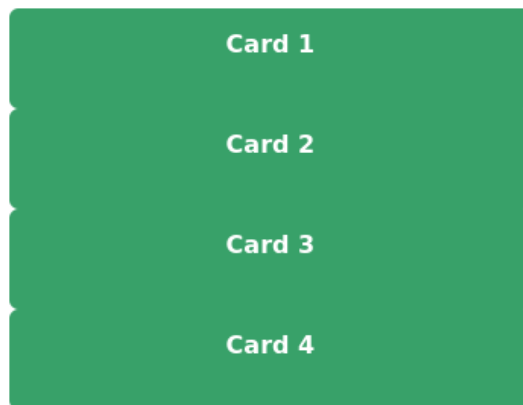
```
* { box-sizing: border-box; }
.container { display: flex; max-width: 1200px; margin: 0 auto; gap: 12px; }
.sidebar { width: 150px; }
.cards { display: flex; flex-wrap: wrap; gap: 10px; flex: 1; }
.card { flex: 1 1 45%; }

@media (max-width: 600px) {
  .sidebar { display: none; }
  .cards { flex-direction: column; }
  .card { flex-basis: 100%; }
}
```

Output - Screen wider than 600px: sidebar visible, cards side by side



Output - Screen 600px or narrower: sidebar hidden, cards stacked vertically



This lets the layout reflow at defined breakpoints - hiding non-essential elements or stacking cards vertically - instead of forcing one fixed layout onto every screen size.

5. Best Practices for a Stable, Responsive Layout

- Apply box-sizing: border-box globally.
- Prefer Flexbox/Grid over floats and absolute positioning for structural layout.
- Use relative units (% , fr , rem , vw) instead of fixed px.
- Design mobile-first and add complexity with min-width media queries.
- Include the viewport meta tag and test at multiple breakpoints.

Question 4

A website requires dynamic content updates without reloading the entire page. This is achieved using the Document Object Model (DOM) and JavaScript DOM manipulation.

1. Concept of the DOM and Its Role

The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of objects - elements, attributes, and text are each nodes connected by parent-child relationships. The browser builds this tree when it parses the HTML.

Its role is to act as the bridge between static HTML and dynamic scripting: JavaScript reads and modifies the in-memory DOM tree, and the browser instantly re-renders the page to reflect those changes - enabling dynamic behaviour without a full reload.

2. How JavaScript Manipulates the DOM

- Selecting elements - `getElementById()`, `querySelector()`, `querySelectorAll()`.
- Changing content - `element.textContent`, `element.innerHTML`, `element.value`.
- Changing styles/classes - `element.style.property`, `classList.add()/remove()/toggle()`.
- Creating/inserting elements - `createElement()`, `appendChild()`, `append()`, `remove()`.
- Responding to actions - `addEventListener('click', handler)` to trigger DOM updates.

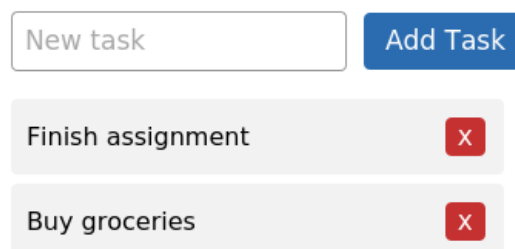
3. Example - Dynamic Content Update Without Reloading

```
const list = document.getElementById('taskList');
const input = document.getElementById('taskInput');

document.getElementById('addBtn').addEventListener('click', () => {
  const text = input.value.trim();
  if (text === '') return;

  const li = document.createElement('li');
  li.textContent = text;
  list.appendChild(li);
  input.value = '';
});
```

Output - Before: list shows "Finish assignment"; input contains "Buy groceries"; after clicking "Add Task":



The screenshot shows a web interface for managing tasks. At the top, there is a text input field labeled "New task" and a blue button labeled "Add Task". Below the input field, there is a list of tasks. The first task is "Finish assignment" and the second task is "Buy groceries". Each task is displayed in a light gray box with a red square containing a white 'X' to its right, indicating a delete function.

Clicking 'Add Task' updates only the list content in the browser's memory - the page never reloads. If the input is empty, nothing is added.

4. Suggested Improvements for Performance and UX

- Batch updates using a DocumentFragment when adding many elements at once.
- Use event delegation (one listener on a parent) instead of one per child, for long lists.
- Debounce input-driven updates (e.g., live search) so they fire after typing pauses.
- Prefer textContent over innerHTML for plain text, to avoid extra parsing and XSS risk.
- For larger apps, consider a framework using a virtual DOM to minimise direct DOM operations.